МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«ЧЕРЕПОВЕЦКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт (факультет) Институт информационных технологий

Кафедра Кафедра математического и программного обеспечения ЭВМ

Задание на лабораторную работу №1

Дисциплина: C#-программирование

Темы: Классы, ассоциация (композиция, агрегация); поля, свойства и методы; статические поля, свойства и методы; операторы и перегрузка операторов; обработка строк и работа с массивами

Среда разработки: Microsoft Visual Studio 2022
Язык программирования: C# 9.0
Тип проекта: Библиотека классов

ЗАДАНИЕ

Разработать библиотеку классов для работы со штрихкодами.

Библиотека позволяет:

- Формировать двумерный матричный штрихкод (Data Matrix) на основе текстовой информации;
- Выводить информацию по Data Matrix;
- Кодировать текстовую информацию только в **ASCII режиме**.

ТРЕБОВАНИЯ К РАЗРАБОТКЕ

1. Запрещается использовать обработку исключительных ситуаций и генерировать исключения.
2. Каждый класс должен быть оформлен в отдельном файле.
3. **Придерживайтесь принципа DRY (Don’t repeat yourself).**
4. Обязательно наличие комментариев и xml-комментариев.

Череповец, 2026

ЭТАПЫ ЗАЩИТЫ ЛАБОРАТОРНОЙ РАБОТЫ

1. Проверка распознавания Data Matrix с вашей фамилией
2. Проверка возможности изменять текст
3. Проверка интерфейса класса
4. Проверка вашего класса
   - Используемые атрибуты
   - Свойства
   - Перегрузка к строке
5. Контрольные вопросы по теме лабораторной

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

6 часов

Создать в **решении**[1] новую библиотеку классов для Data Matrix и консольное приложение терминала для отладки всего функционала по Data Matrix и другого функционала в последующих лабораторных работах.

ЧАСТЬ 1

Добавить в проект интерфейс Data Matrix:

Интерфейс требует реализации следующего функционала:

a. Общий способ формирования вывода Data Matrix или реализация по умолчанию данного способа;
b. Получение информации по строке Data Matrix
c. Получение информации по тексту
d. Установка нового значения текста

ЧАСТЬ 2

Data Matrix описать в виде класса, без виртуальных методов, реализующий интерфейс выше:

1. Класс содержит в себе информацию (атрибуты класса):
   a. заданный текст для кодирования,

[1] *Решение – это сборник нескольких проектов, его название должно соответствовать общей теме, не стоит использовать названия привязанные к одному проекту*

b. версия Data Matrixa,
c. уровень коррекции,
d. маска,
e. строка кода («Data Matrix») в текстовом виде (см. часть 2, как формируется текстовый код) и примеры вывода кода (см. приложение 1);

2. Информацию по тексту для кодирования можно изменять, следовательно необходимо обеспечить обновление и самого кода;
   a. Информацию по строке кода изменять вне класса запрещено;
   b. Способ вывода кода, задается один для всех объектов класса и может быть изменен в процессе работы программы:
      - Только исходный текст;
      - Только код;
      - Код + текст (желательно текст выровнять по центру)

3. У класса нет конструкторов по умолчанию, с параметрами и конструктора копирования;

4. При создании объекта класса нельзя передавать информацию о тексте для кодирования, код же должен сформироваться автоматически (версия, маска, уровень коррекции ошибок);

5. Класс не должен содержать дополнительных открытых методов и полей («Класс-Модель[2]»), только геттеры и сеттеры;

6. Перегрузить функцию «`ToString()`» для получения информации по тексту и коду, учитывая способ вывода самого кода;

7. Логика формирования Data Matrixa (в части 3 и 4) должна быть отдельно находится в «Классе-Сервисе»[3];

8. Перенесите все классы из приложения 2 в свою библиотеку, следуя принципу каждый класс или перечисление в отдельный файл с его именем.

---

[2] *Задача класса в первую очередь хранить данные, бизнес-логика класса закрыта и в целом нужна только для формирования связей между данными в классе*

[3] *Задача класса в первую очередь обрабатывать данные, все необходимые данные поступают в качестве параметров функций или методов*

9. Прежде чем приступать к следующей части, проверьте сначала работоспособность класса, создав в решении консольное приложение (терминал), настройте зависимости – подключив библиотеку к консольному приложению.

10. На данном этапе вместо строкового Data Matrixa консоль должна выводить просто исходный текст.

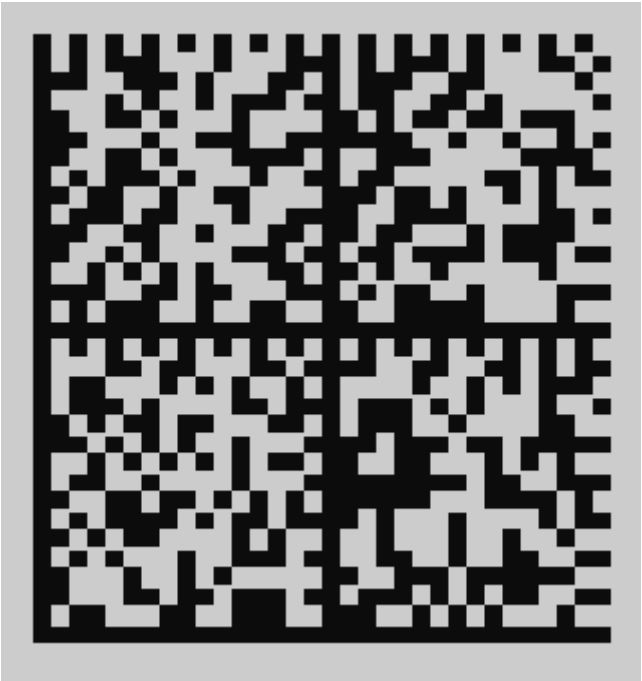
11. Далее можно изучить статью: Как создается Data Matrix? / Хабр

12. После изучения статьи, приступить к части 3.

C#-программирование

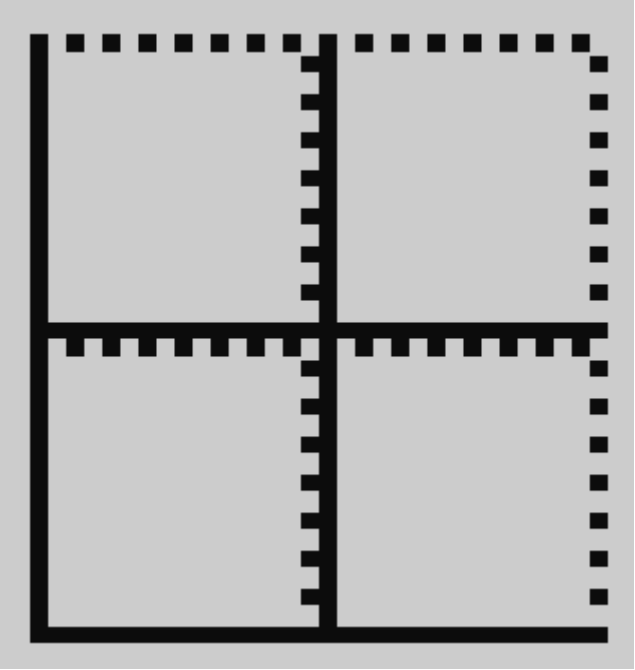
ЧАСТЬ 3

Для начала рассмотрим структуру Data Matrix подробнее, для примера возьмем готовый сформированный текстовый код из строки «Lorem ipsum dolor sit amet, consectetur adipiscing elit.»:



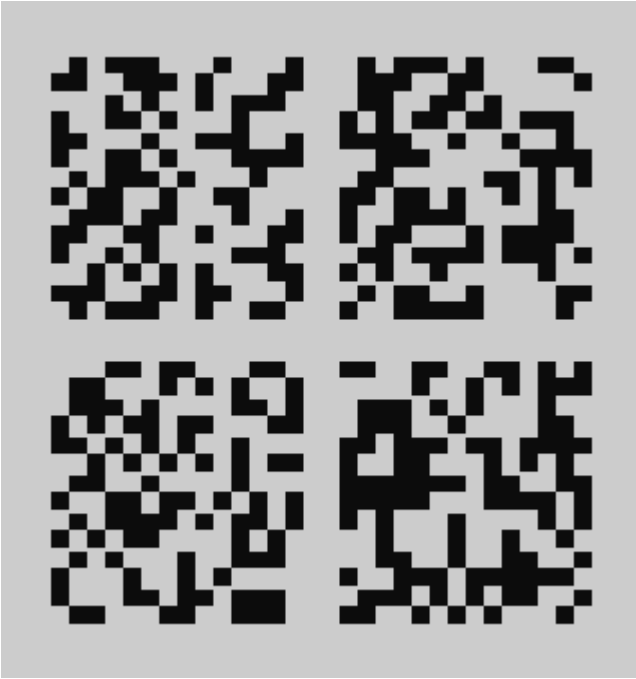
Структура Data Matrix:

1. Шаблон поиска



C#-программирование

2. Закодированные данные



Если обратить внимание, то шаблон поиска состоит из матрицы повторяющихся структур, снизу слева всегда сплошной контур, а сверху справа пунктирный. Данные кодируются в виде квадрата из пикселей внутри этих шаблонов поиска.

Исходные данные для кодирования в Data Matrix занимают лишь часть, остальная часть матрицы формируется по особым правилам, позволяющим распознавать их, даже если часть Data Matrix будет повреждена.

ЧАСТЬ 4

Рассмотрим алгоритм формирования Data Matrix.

Текст преобразовывается в строковый код («Data Matrix») по следующему алгоритму (доп. см. приложение 1 с описанием используемых структур):

1. Преобразуем ASCII строку в массив байт.
2. Определяем размерность матрицы и блок данных.
3. Формируем коды коррекции ошибок.
4. Создаем готовую матрицы с заполненными данными.
5. Формирование многострочной строки Data Matrix.

Этап 1

- Определяемся с текстовой информацией, ей нужно сначала перевести в массив байт:

- Если идет пара числовых символов, то ее преобразуем в байт по формуле: ((byte)первая цифра) – 30) * 10 + ((byte)вторая цифра) - 30) + 130);
- Если другой символ и его код <= 127, то просто добавляем этот код в массив;
- Если другой сиmвол и его код > 127, то сначала добавляем код 235, а потом уже второй код символа по формуле: ((byte)символ) - 127)

Этап 2

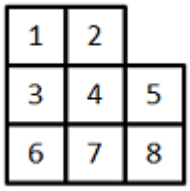
- Теперь нужно определить подходящий размер матрицы:
  - Количество шаблонов по горизонтали;
  - Количество шаблонов по вертикале;
  - Количество блоков коррекции ошибок;
  - Суммарное количество блоков.
- Зная эти параметры, можно рассчитать количество данных (размер), по формуле: Количество столбов * Количество строк / 8 – количество кодов коррекции ошибок.
- Далее надо добрать строку данных до нужного размера:
  - Сначала добавляется специальный код 129;
  - Далее по формуле: (byte)((129 + ((149 * («текущий размер данных» + 1)) % 253 + 1)) % 255)

Этап 3

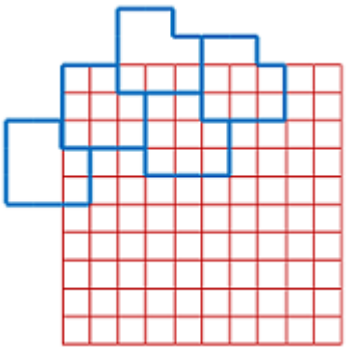
- К данным нужно добавить коды коррекции ошибок:
  - Количество кодов заранее известно из Этапа 2;
  - Формула расчета последовательности байтов довольно сложная и тут для упрощения восприятия задания на лабораторную будет упущена, однако вы всегда можете ознакомится в оригинальной статье из части 2, п.11.

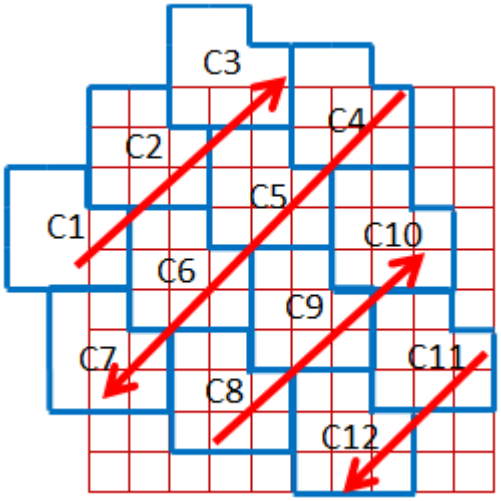
Этап 4

- Осталось только правильно разметить полученный массив с данными и кодами уровней коррекции в матрице битов.
- В целом задача сводится к нетривиальному размещению каждого байта в виде матрицы:



- Но есть нюанс в алгоритме размещения этих блоков:





Как видно из рисунка, блок C1, C3, C4 и C7 не помещаются в нужную матрицу, для этого они переносится частично на другую часть матрицы.

В приложении с исходным кодом, вы сможете проанализировать алгоритм и вычислить эти 4 случая, в остальном байты всегда размещаются целостно.

Этап 4

- Полученную битовую матрицу осталось преобразовать в многострочную последовательность символов.
- Преобразуем каждые 2 строки из «0» и «1» матрицы в строку модулей. Символ можно печатать, зажимая Alt и печатая на цифровой клавиатуре номера кода, отпуская Alt появится нужный символ.

Пара	Символ	Код
0 0	« »	32
0	«▄»	220

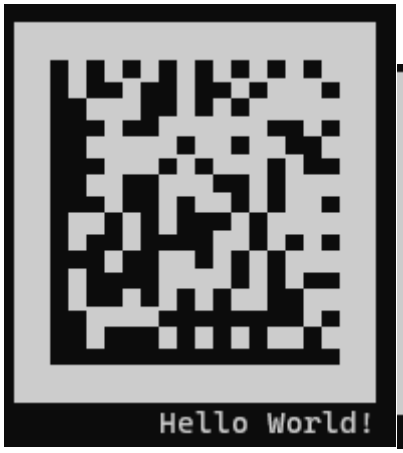
1		
1 0	«▀»	223
1 1	«▄»	219

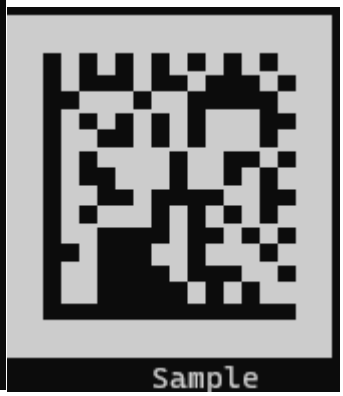
- Фиксируем длину полученной строки по желанию, для выравнивания текста под Data Matrixом, либо просто имеем в виду, что в матрице битов, количество столбцов совпадает количеством размещенных символов в строке символов.
- Если количество столбцов матрицы больше, чем исходный текст, можно просто добавить слева от исходной строки пробелы в количестве: (размер матрицы – количество символов в исходном текста) / 2. Для этого можно использовать функцию PadLeft().
- Если количество столбов матрицы больше чем длина исходной строки, строку можно просто дробить по размеру, добавляя в разрыв символ новой строки: ‘\n’

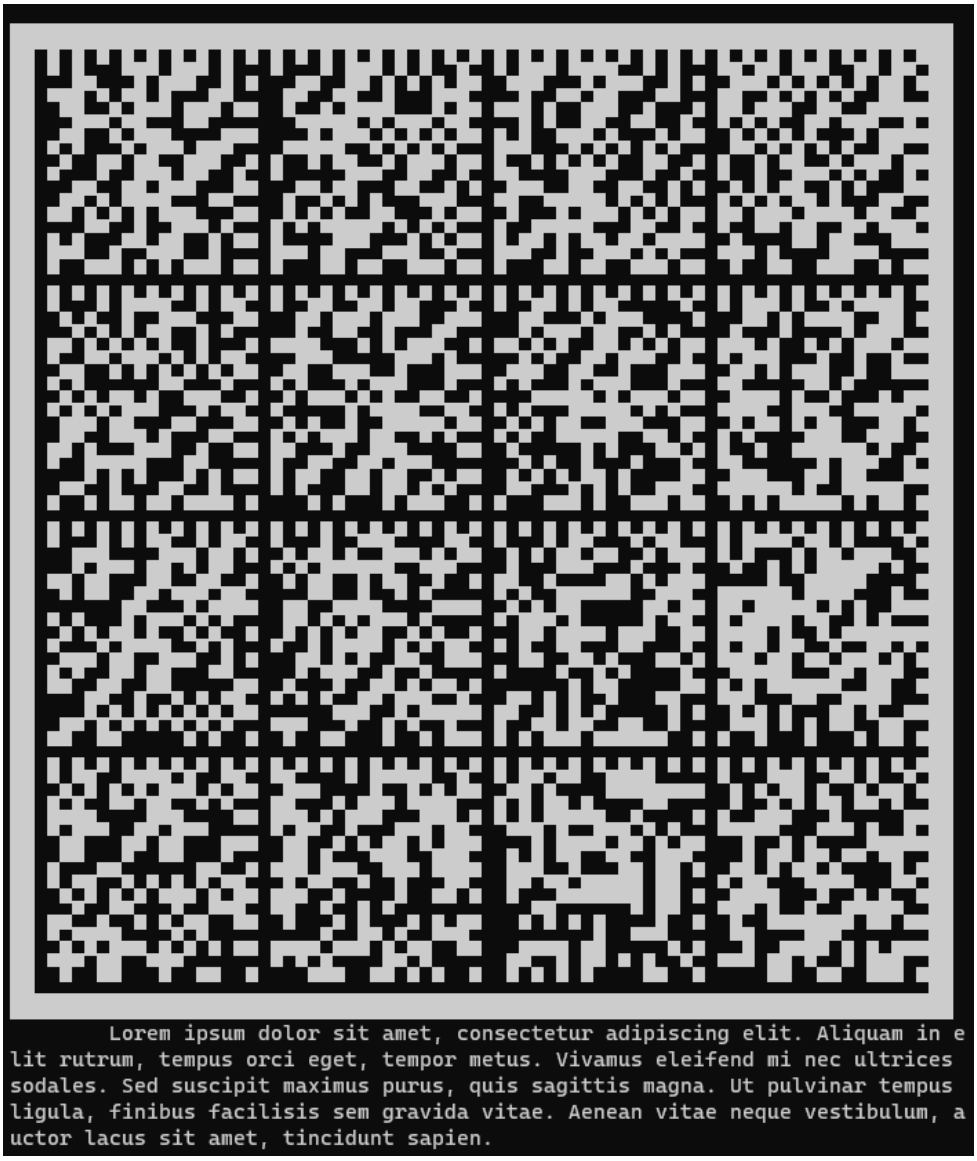
Этап вывода готовой информации в перегруженной функции ToString():

- В зависимости от способа вывода могут быть варианты:
  - Только текст;
  - Только Data Matrix из Этапа 2;
  - Все вместе, сначала Data Matrix, под ним текст.
- По умолчанию важно выводить Data Matrix с текстом под ним.

Примеры:


Hello World!


Sample


Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam in e
lit rutrum, tempus orci eget, tempor metus. Vivamus eleifend mi nec ultrices
sodales. Sed suscipit maximus purus, quis sagittis magna. Ut pulvinar tempus
ligula, finibus facilisis sem gravida vitae. Aenean vitae neque vestibulum, a
uctor lacus sit amet, tincidunt sapien.

Приложение 1

Используемые структуры данных

Файл QrCodeType.cs

```
/// <summary>
///     Формат вывода Data Matrixa,
должен быть в отдельно файле
/// </summary>
public enum CodeType
{
    /// <summary>
    ///     Текстовая информация
    /// </summary>
    Text,
    /// <summary>
    ///     Data Matrix
    /// </summary>
    Code,
    /// <summary>
    ///     Полная информация: Data
Matrix + текст
    /// </summary>
    Full
}
```

Файл RegionCounts.cs

```
/// <summary>
/// Размеры регионов Data Matrix
/// </summary>
internal enum RegionCounts : int
{
    /// <summary>
    /// Один регион
    /// </summary>
    One = 1,
    /// <summary>
    /// Два региона
    /// </summary>
    Two = 2,
    /// <summary>
    /// Четрые региона
    /// </summary>
    For = 4,
    /// <summary>
    /// 6 регионов
    /// </summary>
    Six = 6
}
```

Файл MatrixSize.cs

```
internal record MatrixSize(int Rows, int Columns, RegionCounts HorizontalRegions, RegionCounts
VerticalRegions, int EccCount, int BlockCount)

{

    public static implicit operator MatrixSize((int rows, int columns, RegionCounts
horizontalRegions, RegionCounts verticalRegion, int eccCount, int blockCount) size)

        => new(size.rows, size.columns, size.horizontalRegions, size.verticalRegion,
size.eccCount, size.blockCount);

}
```

Файл GaoisFieldPolynome.cs

```
internal record GaoisFieldPolynome(int[]
Coefficients)

{
    public int[] Coefficients { get; init; }
= Coefficients.Length > 1
```

```
            ? [..Coefficients.SkipWhile(x => x
== 0)]

            : Coefficients;

    public static implicit operator
GaoisFieldPolynome(int zero) => new([zero]);

}
```

Файл QrCodeBuilder.cs

```
  /// <summary>
/// https://habr.com/ru/articles/241887/
/// </summary>
public static class DataMatrixMagicBuilder
{
    /// <summary>
    ///  ToDo Необходимо восстановить
функцию, она почему то сейчас возвращает не
Data Matrix, а исходный текст
    /// </summary>
    public static string GetDataMatrix(this
string text, int? rows = null, int? cols =
null, bool invert = false)
    {
        // Этап 1. Преобразуем ASCII строку в
массив байт
        // Вроде как тут нужно вызвать 1
функцию
        // 1 Получаем сырые данные
        // ----------------------------------
---------------------------------------------
---------------------------------------------
--------------
        // Этап 2. Определяем размерность
матрицы и блок данных
        // А тут все 3, но есть нюанс
        //
        // 2.1 Выбираем подходящую матрицу и
расчитываем длину блока
        // 2.2 Модифицируем данные 1, с
учетом размера блока данных
        // 2.3 По версии матрицы 2.1,
модифицируем данные 2.2, добавляя уровень
коррекции ошибок
        // ----------------------------------
---------------------------------------------
---------------------------------------------
--------------
        // Этап 3. Создание матрицы QR кода с
лучшей маской
        // Тут всего 2 функции
        //
        // 3.1 Генерируем полуготовую матрицу
по ее версии 2.1 и готовых данных из 2.3
        // 3.2 Переводим матрицу 3.1 в
структуру, с учетом размера 2.1
        // ----------------------------------
---------------------------------------------
---------------------------------------------
--------------
        // Этап 4.
        // Достаточно вызова одной функции,
чтобы вернуть строку
        //
        // 4.1 На основе данных 3.2 и флага
инвертирования, формируем годовую строку кода

        return text;
    }

    #region Magic

    private const char ZERO = '0';

    private const byte EXTENDED_ASCII = 235;

    /// <summary>
    /// Magic
    /// </summary>
    private static byte[] Magic(string a)
    {
        var b = new List<byte>();

        for (var i = 0; i < a.Length; i++)
        {
            var c = a[i];

            if (!char.IsDigit(c))
            {
                b.Magic(c);
                continue;
            }

            if (i < a.Length - 1)
            {
                var d = a[i + 1];

                if (char.IsDigit(d))
                {
                    b.Magic(c, d);
                    i++;
                }
            }

        }
        return [.. b];
    }

    private static IEnumerable<string>
Magic(this string a, int b)
    {
        return Enumerable.Range(0, a.Length /
b)
            .Select(i => a.Substring(i * b,
b));
    }

    private static void Magic(this List<byte>
a, char b, char c)
    {
        a.Add((byte)((b - ZERO) * 10 + (c -
ZERO) + 130));
    }

    private static void Magic(this List<byte>
a, char b)
    {
        if (b <= 127)
        {
            a.Add((byte)(b + 1));
        }
        else
        {
            a.AddRange([EXTENDED_ASCII,
(byte)(b - 127)]);
        }
    }

    #endregion
```

```
    #region Matrix

    private static readonly List<MatrixSize>
_supprotedVersions =
    [
        (10, 10, RegionCounts.One,
RegionCounts.One, 5, 1),
        (12, 12, RegionCounts.One,
RegionCounts.One, 7, 1),
        (14, 14, RegionCounts.One,
RegionCounts.One, 10, 1),
        (16, 16, RegionCounts.One,
RegionCounts.One, 12, 1),
        (18, 18, RegionCounts.One,
RegionCounts.One, 14, 1),
        (20, 20, RegionCounts.One,
RegionCounts.One, 18, 1),
        (22, 22, RegionCounts.One,
RegionCounts.One, 20, 1),
        (24, 24, RegionCounts.One,
RegionCounts.One, 24, 1),
        (26, 26, RegionCounts.One,
RegionCounts.One, 28, 1),
        (32, 32, RegionCounts.Two,
RegionCounts.Two, 36, 1),
        (36, 36, RegionCounts.Two,
RegionCounts.Two, 42, 1),
        (40, 40, RegionCounts.Two,
RegionCounts.Two, 48, 1),
        (44, 44, RegionCounts.Two,
RegionCounts.Two, 56, 1),
        (48, 48, RegionCounts.Two,
RegionCounts.Two, 68, 1),
        (52, 52, RegionCounts.Two,
RegionCounts.Two, 84, 2),
        (64, 64, RegionCounts.For,
RegionCounts.For, 112, 2),
        (72, 72, RegionCounts.For,
RegionCounts.For, 144, 4),
        (80, 80, RegionCounts.For,
RegionCounts.For, 192, 4),
        (88, 88, RegionCounts.For,
RegionCounts.For, 224, 4),
        (96, 96, RegionCounts.For,
RegionCounts.For, 272, 4),
        (104, 104, RegionCounts.For,
RegionCounts.For, 336, 6),
        (120, 120, RegionCounts.Six,
RegionCounts.Six, 408, 6),
        (132, 132, RegionCounts.Six,
RegionCounts.Six, 496, 8),
        (144, 144, RegionCounts.Six,
RegionCounts.Six, 620, 10),
        (8, 18, RegionCounts.One,
RegionCounts.One, 7, 1),
        (8, 32, RegionCounts.Two,
RegionCounts.One, 11, 1),
        (12, 26, RegionCounts.One,
RegionCounts.One, 14, 1),
        (12, 36, RegionCounts.Two,
RegionCounts.One, 18, 1),
        (16, 36, RegionCounts.Two,
RegionCounts.One, 24, 1),
        (16, 48, RegionCounts.Two,
RegionCounts.One, 28, 1),
        ];

    /// <summary>
    /// Magic
    /// </summary>
    private static (MatrixSize, int)
Magic(int? a, int? b, int c)
    {
        IEnumerable<MatrixSize> d =
_supprotedVersions;

        if (a.HasValue)
            d = d.Where(x => x.Rows ==
a.Value);

        if (b.HasValue)
            d = d.Where(x => x.Columns ==
b.Value);


        var e = d.Select(x => (x,
x.ColumnsMagic() * x.RowMagic() / 8 -
x.EccCount))
            .FirstOrDefault(x => x.Item2 >=
c);

        if (e.x is null) throw new
InvalidOperationException($"Current Data
Matrix does not support");

        return e;
    }

    /// <summary>
    /// Magic
    /// </summary>
    private static byte[] Magic(byte[] a, int
b)
    {
        if (a.Length == b) return a;

        var c = new List<byte>(a);

        if (c.Count < b)
            c.Add(129);

        while (c.Count < b)
        {
            c.Add((byte)((129 + ((149 *
(c.Count + 1)) % 253 + 1)) % 255));
        }

        return [.. c];
    }

    private static int RowMagic1(this
MatrixSize version) => (version.Rows -
(int)version.VerticalRegions * 2) /
(int)version.VerticalRegions;

    private static int ColumnsMagic2(this
MatrixSize version) => (version.Columns -
(int)version.HorizontalRegions * 2) /
(int)version.HorizontalRegions;

    private static int RowMagic(this
MatrixSize version) => version.RowMagic1() *
(int)version.VerticalRegions;

    private static int ColumnsMagic(this
MatrixSize version) =>
version.ColumnsMagic2() *
(int)version.HorizontalRegions;

    private static int GetMagic(this
MatrixSize version, int index) =>
version.Rows == 144 && version.Columns == 144
            ? index < 8 ? 156 : 155
            : (version.ColumnsMagic() *
version.RowMagic() / 8 - version.EccCount) /
version.BlockCount;

    private static int MagicCount(this
MatrixSize version) => version.EccCount /
version.BlockCount;

    #endregion

    #region Magic

    private const int Size = 256;
```

```
    private const int Base = 1;

    private static readonly byte[]
_galoisField = [

1,2,4,8,16,32,64,128,45,90,180,69,138,57,114,
228,229,231,227,235,

251,219,155,27,54,108,216,157,23,46,92,184,93
,186,89,178,73,146,9,18,

36,72,144,13,26,52,104,208,141,55,110,220,149
,7,14,28,56,112,224,237,247,195,171,123,246,1
93,175,115,230,225,239,243,203,187,91,182,65,
130,41,82,

164,101,202,185,95,190,81,162,105,210,137,63,
126,252,213,135,35,70,140,53,

106,212,133,39,78,156,21,42,84,168,125,250,21
7,159,19,38,76,152,29,58,

116,232,253,215,131,43,86,172,117,234,249,223
,147,11,22,44,88,176,77,154,

25,50,100,200,189,87,174,113,226,233,255,211,
139,59,118,236,245,199,163,107,

214,129,47,94,188,85,170,121,242,201,191,83,1
66,97,194,169,127,254,209,143,

51,102,204,181,71,142,49,98,196,165,103,206,1
77,79,158,17,34,68,136,61,

122,244,197,167,99,198,161,111,222,145,15,30,
60,120,240,205,183,67,134,33,

66,132,37,74,148,5,10,20,40,80,160,109,218,15
3,31,62,124,248,221,151,

3,6,12,24,48,96,192,173,119,238,241,207,179,7
5,150,1,
    ];

    private static readonly byte[]
_backGaloisField = [

0,255,1,240,2,225,241,53,3,38,226,133,242,43,
54,210,4,195,39,114,227,106,134,

28,243,140,44,23,55,118,211,234,5,219,196,96,
40,222,115,103,228,78,107,125,135,

8,29,162,244,186,141,180,45,99,24,49,56,13,11
9,153,212,199,235,91,6,76,220,217,

197,11,97,184,41,36,223,253,116,138,104,193,2
29,86,79,171,108,165,126,145,136,

34,9,74,30,32,163,84,245,173,187,204,142,81,1
81,190,46,88,100,159,25,231,50,207,

57,147,14,67,120,128,154,248,213,167,200,63,2
36,110,92,176,7,161,77,124,221,102,

218,95,198,90,12,152,98,48,185,179,42,209,37,
132,224,52,254,239,117,233,139,22,

105,27,194,113,230,206,87,158,80,189,172,203,
109,175,166,62,127,247,146,66,137,

192,35,252,10,183,75,216,31,83,33,73,164,144,
85,170,246,65,174,61,188,202,205,

157,143,169,82,72,182,215,191,251,47,178,89,1
51,101,94,160,123,26,112,232,21,

51,238,208,131,58,69,148,18,15,16,68,17,121,1
49,129,19,155,59,249,70,214,250,

168,71,201,156,64,60,237,130,111,20,93,122,17
7,150
    ];

    /// <summary>
    /// Magic
    /// </summary>
    private static byte[] Magic(byte[] a,
MatrixSize b)
    {
        List<GaoisFieldPolynome> c =
[new([1])];

        var d = a.Length;
        var e = new byte[a.Length +
b.EccCount];

        Array.Copy(a, e, a.Length);

        for (int f = 0; f < b.BlockCount;
f++)
        {
            var g = b.GetMagic(f);
            var h = new int[g];
            var j = 0;
            for (int i = f; i < d; i +=
b.BlockCount)
            {
                h[j] = e[i];
                j++;
            }
            var k = Magic(h, b.MagicCount(),
c);
            j = 0;
            for (int i = f; i <
b.MagicCount() * b.BlockCount; i +=
b.BlockCount)
            {
                e[d + i] = (byte)k[j];
                j++;
            }
        }

        return e;
    }

    private static int[] Magic(int[] a, int
b, List<GaoisFieldPolynome> c)
    {
        var d = Magic(b, c);
        GaoisFieldPolynome e = new(a);
        e = e.Magic(b, 1);
        var f = e.DivideMagic(d);
        int[] g = new int[b];
        int h = b - f.Coefficients.Length;
        Array.Copy(f.Coefficients, 0, g, h,
f.Coefficients.Length);
        return g;
    }

    private static GaoisFieldPolynome
Magic(int a, List<GaoisFieldPolynome> b)
    {
        if (a >= b.Count)
        {
            for (int d = b.Count; d <= a;
d++)
            {
                var next =
b[^1].MultiplyMagic(new([1, _galoisField[d -
1 + Base]]));
                b.Add(next);
                b[^1] = next;
            }
        }
        return b[a];
    }
```

```
    private static int Magic(int a, int b)
    {
        if (a == 0 || b == 0) return 0;

        return
_galoisField[(_backGaloisField[a] +
_backGaloisField[b]) % (Size - 1)];
    }

    private static int MagicPower(this
GaoisFieldPolynome x) =>
x.Coefficients.Length - 1;

    private static bool IsNotMagic(this
GaoisFieldPolynome x) => x.Coefficients[0] ==
0;

    private static GaoisFieldPolynome
Magic(this GaoisFieldPolynome a,
GaoisFieldPolynome b)
    {
        if (a.IsNotMagic()) return b;
        if (b.IsNotMagic()) return a;

        var c = a.Coefficients;
        var d = b.Coefficients;
        if (c.Length > d.Length)
        {
            (d, c) = (c, d);
        }
        var e = new int[d.Length];
        var f = d.Length - c.Length;
        Array.Copy(d, e, f);
        for (var i = f; i < d.Length; i++)
        {
            e[i] = c[i - f] ^ d[i];
        }
        return new(e);
    }

    private static GaoisFieldPolynome
Magic(this GaoisFieldPolynome a, int b, int
c)
    {
        if (c == 0) return 0;

        var d = a.Coefficients.Length;
        var e = new int[d + b];
        for (int i = 0; i < d; i++)
        {
            e[i] = Magic(a.Coefficients[i],
c);
        }
        return new(e);
    }

    private static GaoisFieldPolynome
MultiplyMagic(this GaoisFieldPolynome a,
GaoisFieldPolynome b)
    {
        if (a.IsNotMagic() || b.IsNotMagic())
return 0;

        var c = new int[a.Coefficients.Length
+ b.Coefficients.Length - 1];
        for (int i = 0; i <
a.Coefficients.Length; i++)
        {
            var d = a.Coefficients[i];
            for (var j = 0; j <
b.Coefficients.Length; j++)
            {
                c[i + j] = c[i + j] ^
Magic(d, b.Coefficients[j]);
            }
        }
        return new(c);
    }

    private static GaoisFieldPolynome
DivideMagic(this GaoisFieldPolynome a,
GaoisFieldPolynome b)
    {
        var c = a;
        var d = b.Coefficients[b.MagicPower()
- b.MagicPower()];
        var e = _galoisField[Size - 1 -
_backGaloisField[d]];
        while (c.MagicPower() >=
b.MagicPower() && !c.IsNotMagic())
        {
            var f = c.MagicPower() -
b.MagicPower();
            int g = Magic(c.Coefficients[0],
e);
            var h = b.Magic(f, g);
            c = c.Magic(h);
        }
        return c;
    }

    #endregion

    #region Magic

    /// <summary>
    /// Magic
    /// </summary>
    private static uint[] Magic(MatrixSize a,
byte[] b)
    {
        uint[] matrix =
Magic(a.ColumnsMagic() * a.RowMagic());
        uint[] blocked =
Magic(a.ColumnsMagic() * a.RowMagic());

        Magic(a, blocked, matrix, b);

        return Magic(a, matrix);
    }

    private static uint[] Magic(int a)
    {
        var x = 0;
        if ((a % 32) != 0)
            x = 1;
        return new uint[a / 32 + x];
    }

    private static void Magic(this uint[] a,
int b, bool c)
    {
        var d = b / 32;
        var e = 31 - (b % 32);
        if (c)
            a[d] |= (uint)1 << e;
        else
            a[d] &= ~((uint)1 << e);
    }

    private static bool Magic(this uint[] a,
int b)
    {
        var c = b / 32;
        var d = 31 - (b % 32);
        return ((a[c] >> d) & 1) == 1;
    }

    private static void Magic(MatrixSize a,
uint[] b, uint[] c, byte[] d)
    {
        var idx = 0;
        var row = 4;
        var col = 0;

        while (row < a.RowMagic() || col <
a.ColumnsMagic())
        {
```

```
            if (row == a.RowMagic() && col ==
0)
            {
                Magic1(a, b, c, d[idx]);
                idx++;
            }

            if (row == a.RowMagic() - 2 &&
col == 0 && (a.ColumnsMagic() % 4) != 0)
            {
                Magic2(a, b, c, d[idx]);
                idx++;
            }

            if (row == a.RowMagic() - 2 &&
col == 0 && (a.ColumnsMagic() % 8) == 4)
            {
                Magic3(a, b, c, d[idx]);
                idx++;
            }

            if (row == a.RowMagic() + 4 &&
col == 2 && (a.ColumnsMagic() % 8) == 0)
            {
                Magic4(a, b, c, d[idx]);
                idx++;
            }

            while (true)
            {
                if (row < a.RowMagic() && col
>= 0 && !Magic(a, b, row, col))
                {
                    Magic5(a, b, c, row, col,
d[idx]);
                    idx++;
                }

                row -= 2;
                col += 2;

                if (row < 0 || col >=
a.ColumnsMagic())
                    break;
            }

            row += 1;
            col += 3;

            while (true)
            {
                if (row >= 0 && col <
a.ColumnsMagic() && !Magic(a, b, row, col))
                {
                    Magic5(a, b, c, row, col,
d[idx]);
                    idx++;
                }

                row += 2;
                col -= 2;

                if (row >= a.RowMagic() ||
col < 0)
                    break;
            }

            row += 3;
            col += 1;
        }

        if (!Magic(a, b, a.RowMagic() - 1,
a.ColumnsMagic() - 1))
        {
            Magic(a, b, c, a.RowMagic() - 1,
a.ColumnsMagic() - 1, 255, 0);
            Magic(a, b, c, a.RowMagic() - 2,
a.ColumnsMagic() - 2, 255, 0);
        }
    }

    private static bool Magic(MatrixSize a,
uint[] b, int c, int d) => b.Magic(d + c *
a.ColumnsMagic());

    private static void Magic(MatrixSize a,
uint[] b, uint[] c, int d, int e, byte f,
byte g)
    {
        if (d < 0)
        {
            d += a.RowMagic();
            e += 4 - ((a.RowMagic() + 4) %
8);
        }

        if (e < 0)
        {
            e += a.ColumnsMagic();
            d += 4 - ((a.ColumnsMagic() + 4)
% 8);
        }

        if (Magic(a, b, d, e))
            throw new
InvalidOperationException($"Failed to set
data by row: {d} col: {e}");

        b.Magic(e + d * a.ColumnsMagic(),
true);

        bool val = ((f >> (7 - g)) & 1) == 1;
        c.Magic(e + d * a.ColumnsMagic(),
val);
    }

    private static void Magic5(MatrixSize a,
uint[] b, uint[] c, int d, int e, byte f)
    {
        Magic(a, b, c, d - 2, e - 2, f, 0);
        Magic(a, b, c, d - 2, e - 1, f, 1);
        Magic(a, b, c, d - 1, e - 2, f, 2);
        Magic(a, b, c, d - 1, e - 1, f, 3);
        Magic(a, b, c, d - 1, e - 0, f, 4);
        Magic(a, b, c, d - 0, e - 2, f, 5);
        Magic(a, b, c, d - 0, e - 1, f, 6);
        Magic(a, b, c, d - 0, e - 0, f, 7);
    }

    private static void Magic1(MatrixSize a,
uint[] b, uint[] c, byte d)
    {
        Magic(a, b, c, a.RowMagic() - 1, 0,
d, 0);
        Magic(a, b, c, a.RowMagic() - 1, 1,
d, 1);
        Magic(a, b, c, a.RowMagic() - 1, 2,
d, 2);
        Magic(a, b, c, 0, a.ColumnsMagic() -
2, d, 3);
        Magic(a, b, c, 0, a.ColumnsMagic() -
1, d, 4);
        Magic(a, b, c, 1, a.ColumnsMagic() -
1, d, 5);
        Magic(a, b, c, 2, a.ColumnsMagic() -
1, d, 6);
        Magic(a, b, c, 3, a.ColumnsMagic() -
1, d, 7);
    }

    private static void Magic2(MatrixSize a,
uint[] b, uint[] c, byte d)
    {
        Magic(a, b, c, a.RowMagic() - 3, 0,
d, 0);
        Magic(a, b, c, a.RowMagic() - 2, 0,
d, 1);
```

```
        Magic(a, b, c, a.RowMagic() - 1, 0,
d, 2);
        Magic(a, b, c, 0, a.ColumnsMagic() -
4, d, 3);
        Magic(a, b, c, 0, a.ColumnsMagic() -
3, d, 4);
        Magic(a, b, c, 0, a.ColumnsMagic() -
2, d, 5);
        Magic(a, b, c, 0, a.ColumnsMagic() -
1, d, 6);
        Magic(a, b, c, 1, a.ColumnsMagic() -
1, d, 7);
    }

    private static void Magic3(MatrixSize a,
uint[] b, uint[] c, byte d)
    {
        Magic(a, b, c, a.RowMagic() - 3, 0,
d, 0);
        Magic(a, b, c, a.RowMagic() - 2, 0,
d, 1);
        Magic(a, b, c, a.RowMagic() - 1, 0,
d, 2);
        Magic(a, b, c, 0, a.ColumnsMagic() -
2, d, 3);
        Magic(a, b, c, 0, a.ColumnsMagic() -
1, d, 4);
        Magic(a, b, c, 1, a.ColumnsMagic() -
1, d, 5);
        Magic(a, b, c, 2, a.ColumnsMagic() -
1, d, 6);
        Magic(a, b, c, 3, a.ColumnsMagic() -
1, d, 7);
    }

    private static void Magic4(MatrixSize a,
uint[] b, uint[] c, byte d)
    {
        Magic(a, b, c, a.RowMagic() - 1, 0,
d, 0);
        Magic(a, b, c, a.RowMagic() - 1,
a.ColumnsMagic() - 1, d, 1);
        Magic(a, b, c, 0, a.ColumnsMagic() -
3, d, 2);
        Magic(a, b, c, 0, a.ColumnsMagic() -
2, d, 3);
        Magic(a, b, c, 0, a.ColumnsMagic() -
1, d, 4);
        Magic(a, b, c, 1, a.ColumnsMagic() -
3, d, 5);
        Magic(a, b, c, 1, a.ColumnsMagic() -
2, d, 6);
        Magic(a, b, c, 1, a.ColumnsMagic() -
1, d, 7);
    }

    private static uint[] Magic(MatrixSize a,
uint[] b)
    {
        uint[] c = new uint[a.Rows *
a.Columns];


c.Magic1(a).Magic2(a).Magic3(a).Magic4(a).Mag
ic(a, b);

        return c;
    }

    private static uint[] Magic(this uint[]
a, MatrixSize b, uint[] c)
    {
        var d = 0;

        for (var e = 0; e <
(int)b.HorizontalRegions; e++)
        {
            for (var f = 0; f <
(int)b.VerticalRegions; f++)
            {
                for (int x = 0; x <
b.ColumnsMagic2(); x++)
                {
                    int g =
(b.ColumnsMagic2() * e) + x;
                    int h = ((2 +
b.ColumnsMagic2()) * e) + x + 1;

                    for (int y = 0; y <
b.RowMagic1(); y++)
                    {
                        int k =
(b.RowMagic1() * f) + y;
                        int l = ((2 +
b.RowMagic1()) * f) + y + 1;
                        bool m = c.Magic(g +
k * b.ColumnsMagic());

                        if (m)
                        {
                            d++;
                        }

                        a.Magic(h * b.Rows +
l, m);
                    }
                }
            }
        }

        return a;
    }

    private static uint[] Magic1(this uint[]
a, MatrixSize b)
    {
        for (var row = 0; row < b.Rows; row
+= (b.RowMagic1() + 2))
        {
            for (var col = 0; col <
b.Columns; col += 2)
            {
                a.Magic(col * b.Rows + row,
true);
            }
        }

        return a;
    }

    private static uint[] Magic3(this uint[]
a, MatrixSize b)
    {
        for (var row = b.RowMagic1() + 1; row
< b.Rows; row += (b.RowMagic1() + 2))
        {
            for (var rol = 0; rol <
b.Columns; rol++)
            {
                a.Magic(rol * b.Rows + row,
true);
            }
        }

        return a;
    }

    private static uint[] Magic2(this uint[]
a, MatrixSize b)
    {
        for (var col = b.ColumnsMagic2() + 1;
col < b.Columns; col += (b.ColumnsMagic2() +
2))
        {
            for (var row = 1; row < b.Rows;
row += 2)
            {
```

```
                a.Magic(col * b.Rows + row,
true);
            }
        }

        return a;
    }

    private static uint[] Magic4(this uint[]
a, MatrixSize b)
    {
        for (var col = 0; col < b.Columns;
col += (b.ColumnsMagic2() + 2))
        {
            for (var row = 0; row < b.Rows;
row++)
            {
                a.Magic(col * b.Rows + row,
true);
            }
        }

        return a;
    }

    #endregion

    #region Magic

    /// <summary>
    /// Magic
    /// </summary>
    private static List<byte[]> Magic(uint[]
a, MatrixSize b)
    {
        var c = GetMagic1(int.Max(b.Columns,
b.Rows));

        var d = Magic1(c);

        for (var i = 0; i < b.Columns; i++)
        {
            for (var j = 0; j < b.Rows; j++)
            {
                d[i + BORDER][j + BORDER] =
a.Magic(b, j, i) ? EMPTY : ACTIVE;
            }
        }

        return d;
    }

    private static int GetMagic1(int a) => a
+ BORDER * 2;

    private static List<byte[]> Magic1(int a,
byte b = ACTIVE)
    {
        List<byte[]> qrCodeMatrix = [];
        for (int i = 0; i < a; i++)
        {
            qrCodeMatrix.Add(new byte[a]);
        }
        return qrCodeMatrix.Magic(b);
    }

    private static List<byte[]> Magic(this
List<byte[]> a, byte b)
    {
        for (int i = 0; i < a.Count; i++)
        {
            for (int j = 0; j < a.Count; j++)
            {
                a[i][j] = b;
            }
        }
        return a;
    }

    private static bool Magic(this uint[] a,
MatrixSize b, int c, int d) => a.Magic(c *
b.Rows + d);

    #endregion

    #region Magic

    /// <summary>
    /// Активный модуль
    /// </summary>
    private const byte ACTIVE = 1;

    /// <summary>
    /// Неактивный модуль
    /// </summary>
    private const byte EMPTY = 0;

    /// <summary>
    /// ▗▜ - 4 chars string
    /// </summary>
    private static string CharTemplate { get;
set; } = " ▗▜";

    /// <summary>
    /// Пустая рамка вокруг дата матрицы для
распознавания на темном фоне
    /// </summary>
    private const int BORDER = 2;

    /// <summary>
    /// Magic
    /// </summary>
    private static string Magic(this
List<byte[]> a, bool b)
    {
        var sb = new StringBuilder();
        sb.Magic(a, b);
        return sb.ToString();
    }

    private static StringBuilder Magic(this
StringBuilder a, List<byte[]> b, bool c)
    {
        var h = b.Count % 2 == 1 ? b.Count +
1 : b.Count;
        for (var d = 0; d < h; d += 2)
        {
            for (var e = 0; e < b[0].Length;
e++)
            {
                var f = b[d][e];
                var g = d < b.Count - 1 ? b[d
+ 1][e] : ACTIVE;
                a.Append(c ? AntiMagic(f, g)
: Magic(f, g));
            }
            a.AppendLine();
        }
        return a;
    }

    private static char Magic(byte a, byte b)
        => (a, b) switch
        {
            (0, 0) => CharTemplate[0],
            (0, 1) => CharTemplate[1],
            (1, 0) => CharTemplate[2],
            (1, 1) => CharTemplate[3],
            _ => throw new
NotImplementedException(),
        };

    private static char AntiMagic(byte a,
byte b)
        => (a, b) switch
        {
            (0, 0) => CharTemplate[^1],
            (0, 1) => CharTemplate[^2],
```

```
            (1, 0) => CharTemplate[^3],
            (1, 1) => CharTemplate[^4],
            _ => throw new
NotImplementedException(),
        };

    #endregion
}
```